AutoGrader: A Rule-Driven Evaluator for C Programs

Team (4): Ratnesh Pandey (2611MC19) · Harsh Yadav (2611MC04) · Souradeep Mondal (2611MC15) · Saurav Kumar (2611MC08)

1. Project summary

AutoGrader is an unattended agent that grades a student's C program against an instructor-supplied rules file. The instructor provides a C source file, a rules file (JSON) and test cases. The agent compiles the program, runs it under time and memory limits, scores it on nine weighted metrics, and writes a reproducible report in JSON and plain text. The same submission and the same rules file are meant to give the same result each time; the SHA-256 hash of the rules file is stored in every report to support this.

Since the last report the project has moved from a first prototype to a working end-to-end system. All nine metrics required by the assignment now exist in code: correctness, syntax, structure, modularity, logical (edge cases), complexity, memory safety, readability and originality. The remaining week is to be spent on making the agent work for more complex programs (not only “add two integers”), on safer execution, and on the final report, README and demo video.

2. System architecture

Figure 1 shows the architecture. The agent works in two coordinated phases, followed by a collation step that produces the report.

- **Inputs.** the student's C file, the instructor's rules.json (metric weights, time and memory limits, sample input, LLM settings, per-metric parameters) and the test sets in tests/<problem>/ as .in/.out pairs.
- **Phase 1 — Sanity coordinator (phase1_coordinator.py).** loads the rules file and computes its SHA-256 hash; checks that the metric weights sum to 1.00; copies the submission into a temporary sandbox folder; checks statically that main(), a reader (e.g. scanf), a writer (e.g. printf) and a return statement exist; compiles with GCC (-O2 -Wall -Wextra -std=c11); and runs a smoke test with the sample input under a 1-second wall-clock limit and a 64 MB memory limit. It then decides which analyzers to run. If compilation fails or the smoke run times out (suspected infinite loop), the execution-based analyzers are skipped and recorded with a score of 0 and the reason.
- **Phase 2 — Expert analyzers (phase2_dispatcher.py, analyzers.py).** each metric in the rules file names an analyzer, which is looked up in a registry and run in parallel threads. Execution-based analyzers (io_tests, edge_case_runner, complexity_estimator, sanitizer_run) need the compiled binary. Source-only analyzers (compiler_warnings, static_analysis, function_split, llm_review, similarity_flag) still run when compilation fails. Every analyzer returns a score between 0 and 1, evidence and feedback text.
- **LLM helper (llm_test_generator.py).** uses a local Ollama server (llama3.1, temperature 0, seed 42) to generate a test set and targeted edge cases from the problem description and the student's code, together with a Python oracle that computes the expected outputs. Generated tests are cached on disk (tests/add/, tests/.edge_cache/) so later runs are repeatable, and built-in fallback tests are used if Ollama is not available.

- **Collation and report (autograder.py).** adds up the weighted scores to a grade out of 100, lists the failing inputs with expected and actual output, produces suggestions, records usefulness and limitations of the evaluation and the versions of the tools used, and writes reports/<name>.json and reports/<name>.txt. main.py is the command-line entry point.

Project layout: main.py; agent/ (phase1_coordinator.py, phase2_dispatcher.py, analyzers.py, llm_test_generator.py, autograder.py); rules/ (add_rules.json); tests/ (add/ and .edge_cache/); submissions/ (sample C programs); reports/ (generated reports); README.md. In total about 1,900 lines of Python.

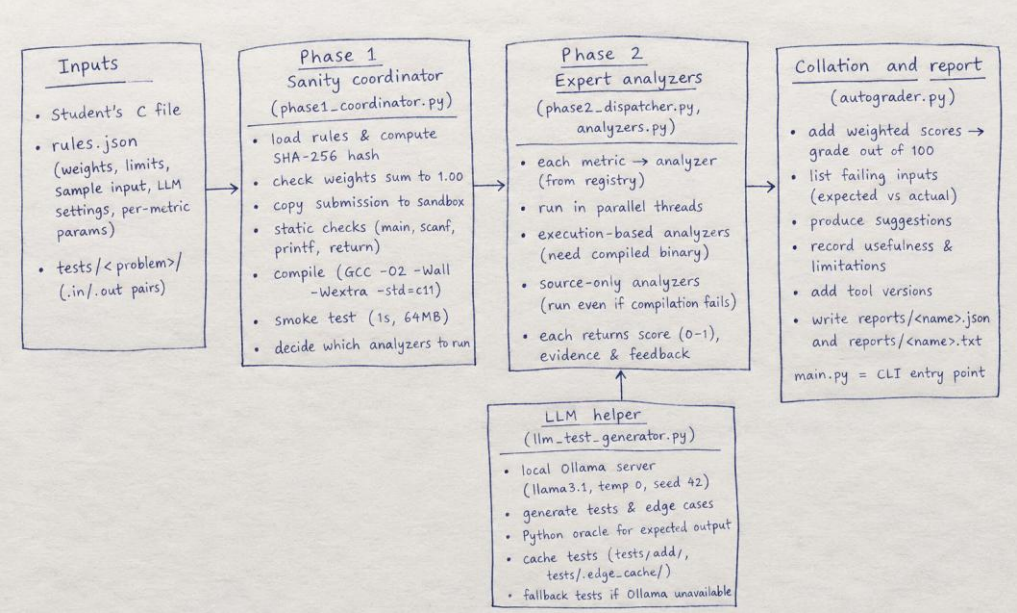


Figure 1: AutoGrader architecture — inputs, Phase 1 sanity coordinator, Phase 2 expert analyzers and report collation.

3. Work completed so far

Weeks 1–2. We studied the assignment, decided the design and built the first prototype: compilation with GCC, a wall-clock timeout with POSIX resource limits, JSON-configured weights and a JSON report.

Week 3 (since the last report).

- Reorganised the code into the two-phase architecture above, with a registry of analyzers so that a new metric can be added by writing one function and naming it in the rules file.
- Implemented all nine metrics, with the weights, limits and parameters read from rules/add_rules.json.
- Added test sets: ten input/output cases for "add two integers" (including 32-bit boundary values such as 2147483647 + 1), plus six targeted edge cases generated for each submission and cached.
- Added a rules check: the rules file must exist and the weights must sum to 1.00, otherwise a clear error is raised; an unknown analyzer name is reported in the output instead of crashing the run.
- Added sample submissions for every path: correct and modular (add_modular.c), logically flawed (add.c, which overflows), near-copy (add_copied.c), runtime crash (add_crash.c), infinite loop (add_infinite_loop.c) and compile error (add_syntax_error.c).
- The agent now survives bad submissions: the infinite-loop sample is killed by the timeout and its process group is terminated; the crash sample is recorded and the remaining analyzers still run.

- Report output now contains the grade, one line per metric, the failing inputs with expected and actual output, suggestions to raise the grade, four usefulness examples and four limitations, and the versions of GCC, Python and other tools.

4. Metrics and scoring

The weights below come from rules/add_rules.json. Each analyzer returns a score from 0 to 1, and the grade is the sum of score × weight × 100.

- **Correctness (20%):** fraction of the ten test cases whose output matches the expected output.
- **Syntax (10%):** GCC with -Wall -Wextra; 0.10 is deducted per warning, and the score is 0 if compilation fails.
- **Structure (10%):** cyclomatic complexity and function length from the lizard tool, plus simple structure checks.
- **Modularity (10%):** number of functions found, compared with the minimum set in the rules (two for this problem).
- **Logical (10%):** fraction of the generated edge cases (negatives, zero, overflow, whitespace) that pass.
- **Complexity (10%):** estimated time and space complexity from timing at several input sizes and a loop/allocation scan, compared with the complexity required by the rules (O(1) here).
- **Memory safety (10%):** the program is rebuilt with AddressSanitizer and UndefinedBehaviorSanitizer and run on several inputs.
- **Readability (10%):** score of 0–10 from the LLM review of naming, comments and formatting; a fixed rule-based check is used if the LLM is not reachable.
- **Originality (10%):** similarity of the normalised source to the other .c files in the submissions/ folder; a program at or above 0.85 similarity is flagged and scores 0 for this metric.

5. Testing and current results

We ran the agent on the six sample submissions; the generated reports are saved in the reports/ folder. The grades are:

- **add_modular.c — 97 / 100.** Passes 10/10 tests and 6/6 edge cases, has a helper function and is clean on every metric except syntax, where three compiler warnings cost 3 points.
- **add.c — 65 / 100.** Passes 8/10 tests. It fails 2147483647 + 1 and -2147483648 + (-1) because of signed integer overflow (got -2147483648 instead of 2147483648). It also loses points for having only main(), single-letter variable names, no comments, and an originality flag (similarity 0.94 with another file in the folder).
- **add_copied.c — 65 / 100.** A copy of add.c with only the variable names and spacing changed. The same bugs give the same score, and the originality metric flags it (similarity 0.92).
- **add_crash.c — 56 / 100.** The program exits with an error, so 0/10 tests pass and the logical metric is 0, but the compile-based and source-based metrics are still scored.
- **add_infinite_loop.c — 36 / 100.** The smoke run times out and the report carries the flag suspected_infinite_loop. Correctness, logical, complexity and memory safety are skipped with the reason recorded; source-only metrics are still scored.

- **add_syntax_error.c — 16 / 100.** Compilation fails with 2 errors. Execution-based metrics are skipped and the syntax metric is 0.

These runs show the main paths (correct, wrong answer, copy, crash, timeout, compile error) behave as designed and that the grade changes with the weights in the rules file. They are evidence for one simple problem only; they do not show that the agent is correct for other problems.

6. What is still incomplete

- Only one problem ("add two integers") has been tested. Several parts of the code still assume it: the fallback tests and oracle, the sample input, the inputs used by the complexity and sanitizer analyzers, and the wording of some suggestions. A complex program (arrays, strings, recursion, dynamic memory) is not yet supported.
- Execution is not a secure sandbox. The agent uses a temporary folder, a timeout and POSIX resource limits (CPU time, address space), but there is no Docker container or virtual machine, no network isolation and no low-privilege account. Unknown submissions should not be run on a sensitive machine.
- Memory safety: the sample reports were produced with GCC 6.3.0, which has no sanitizer libraries, so the agent used its static fallback (a scan for unsafe functions such as gets and strcpy and for malloc/free mismatches). Real AddressSanitizer/UndefinedBehaviorSanitizer runs still have to be tested on a Linux toolchain.
- Readability and test generation depend on a local Ollama server. When it is offline, the agent uses a deterministic rule-based readability check and built-in fallback tests, so those results are less rich. The sample reports show the readability fallback in use.
- The complexity estimate is a heuristic (timing plus a loop/allocation scan). For a program that takes one input size it cannot tell O(N) from $O(N^2)$, and operating-system timing noise can mislead it.
- Originality compares token text with local .c files only, using a simple similarity ratio. It does not search online sources or prove copying. Short programs look alike: add_syntax_error.c was flagged at 0.94, which is a false-positive risk.
- There is no automated regression-test script in the repository yet, and rules validation is limited to the file check, the weight sum and unknown analyzer names.

7. Member-wise work and responsibility

The component split is below. The ZIP does not contain member-level commit history, so the named contributions follow the group work split and should be checked against what each person actually did.

Harsh Yadav (2611MC04) — execution path and safety.

- **Done:** Phase 1 coordinator — sandbox folder, GCC compile, smoke run, timeout and memory limits, infinite-loop and crash handling.
- **Remaining:** test the sanitizer run on Linux, try Docker isolation with network disabled and a non-root user, and document what is and is not isolated.

Souradeep Mondal (2611MC15) — tests and correctness.

- **Done:** ten-case test set with boundary values, the correctness and edge-case analyzers, and the LLM test generator with caching.

- **Remaining:** tests and expected outputs for the new, more complex problems; make the fallback tests and oracle part of the rules file.

Ratnesh Pandey (2611MC19) — rules and metrics.

- **Done:** rules JSON design with weights and parameters, and the metric analyzers (syntax, structure, modularity, complexity, readability, originality).
- **Remaining:** rules files for new problems, a better size-based complexity estimate, and clearer metric explanations in the report.

Saurav Kumar (2611MC08) — integration and reporting.

- **Done:** main.py, the dispatcher that runs the analyzers in parallel and collates scores, the JSON and text reports, the README and the sample submissions.
- **Remaining:** regression-test script, final README and setup guide, demo video and final documentation.

Team work. All four members review integrated changes, run the same samples before merging, and check that the score printed in the console agrees with the report file. Claims about security and evaluation in the final write-up must match what has actually been tested.

8. Plan for the remaining work

The aim is to show that the agent works beyond the simple add program, while keeping the working end-to-end path stable.

Complex programs

- Add at least two more complex problems, each with its own rules file, test set and good/bad sample submissions. Candidates: sorting an array of N integers, string handling (reverse or palindrome), recursion (factorial or Fibonacci) and a program using dynamic memory such as a linked list.
- Remove the add-specific assumptions: move the sample input, fallback tests, oracle, complexity input sizes and sanitizer inputs into the rules file so each problem supplies its own.
- Improve the complexity estimate for programs whose input size N grows (for example N = 1,000, 10,000 and 100,000) so that O(N) and $O(N^2)$ can be told apart; keep labelling it as an estimate.
- Add seeded “bad” samples for the new problems, such as out-of-bounds array access, a memory leak and a slow quadratic loop, to check that the memory-safety and complexity metrics react.

Safety and reliability

- Run the AddressSanitizer/UndefinedBehaviorSanitizer analyzer on a Linux toolchain (for example WSL) and record the results.
- Try running submissions in a Docker container with the network disabled and a non-root user; if it cannot be completed in time, document clearly that isolation is not provided.
- Add an automated regression script that runs all sample submissions and compares the grades with the recorded values; make rules validation stricter.

Documentation and demo

- Finish the README with installation steps (Python packages requests and lizard, GCC, optional Ollama) and example commands.

- Record the 15–20 minute demo video showing the architecture, one correct, one incorrect and one failing/timeout example, and at least one complex program.
- Prepare the final report with the usefulness and limitations section and the GitHub repository.

Evidence to finish the week: a clean run from the README on a fresh machine; reports for the old and new problems; regression results recorded; final code, report and demo ready for submission.

9. Priorities if time becomes tight

- **First:** one new complex problem working end to end with its own rules file and tests, and the add-specific assumptions removed.
- **Second:** regression script and a Linux run of the sanitizer analyzer.
- **Third:** README, final report and demo video.
- **Only after these:** Docker isolation and richer originality checks. If Docker is not finished, the report will state that the runner is not isolated.

10. Risks and how we will handle them

- **Complex programs expose hidden assumptions.** Parts of the code were written around one problem. Response: do the generalisation first and test it on the old samples so nothing breaks.
- **Unsafe submitted code.** Process limits alone do not stop a hostile program from affecting the host. Response: use a container where feasible, and do not claim isolation unless it has been tested.
- **LLM unavailable or variable.** Test generation and readability review depend on a local model. Response: temperature 0 and a fixed seed, cached tests, and a deterministic fallback that is shown in the report.
- **Misleading heuristic scores.** Complexity, readability and similarity values can be misread as verdicts. Response: label them as estimates and treat the similarity flag as a prompt for human review.
- **Time remaining.** Only one week is left. Response: finish the end-to-end path first and keep advanced features optional.

11. Usefulness and limitations

Possible real-life uses

- A programming lab with hundreds of students can run the same tests and scoring rules on every submission and get consistent, explainable grades.
- Students see the failing input with expected and actual output, plus concrete suggestions, instead of only a final mark.
- Instructors can review only the pairs flagged by the similarity check instead of every submission.
- Teams can use the same agent as a quality gate before merging code, since it checks warnings, complexity and memory safety.

Limitations, with examples from our runs

- Passing tests is not proof of correctness: add.c passes 8 of 10 tests and a hidden input outside the test set could still break it.
- A similarity flag is not proof of copying: very short programs look alike, and add_syntax_error.c was flagged at 0.94 although it is a broken program.
- A renamed or restructured copy can score below the threshold, so a low similarity does not rule out copying.
- The complexity and readability scores are heuristics and can misjudge valid code.
- A process limit is not a secure sandbox; untrusted code needs stronger isolation before real use.

12. Conclusion

By the end of Week 3 AutoGrader is a working two-phase agent that grades a C submission on nine rule-driven metrics, survives compile errors, crashes and infinite loops, and writes JSON and text reports. It has been demonstrated on one simple problem and six sample submissions. The last week will focus on making it handle more complex programs, testing the memory-safety checks on Linux, attempting safer execution, and completing the README, final report and demo video. Anything that remains a rough estimate or is not isolated will be described as such in the final report.